VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

PRERITH M SHETTY (1WA24CS219)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by PRERITH M SHETTY (1WA24CS219), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Prof. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	1-7
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	8-19
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	20-23
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	24-35
5.	Write a C program to simulate a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem.	36-43
6.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	44-52
7.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	53-60
8.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	61-67

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include <stdio.h>
int main() {
   printf("Name:Prerith M Shetty\n" "USN:1WA24CS219\n"
   "PRG:FCFS\n"); int n, i;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n];
   int current_time = 0;
   float avgTAT = 0, avgWT = 0;

   for(i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
      scanf("%d %d", &AT[i], &BT[i]);
   }
   for(i = 0; i < n; i++) {
      if(current_time < AT[i])
         current_time = AT[i];
      RT[i] = current_time - AT[i];
      CT[i] = current_time + BT[i];
```

```
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        current_time = CT[i];
        avgTAT += TAT[i];
        avgWT += WT[i];
    }
   printf("\n%-5s %-12s %-10s %-15s %-15s %-12s %-12s\n",
"P","ArrivalTime","BurstTime","CompletionTime","TurnAroundTime","WaitingTime","Response
Time");

    for(i=0;i<n;i++)
        printf("%-5d %-12d %-10d %-15d %-15d %-12d %-12d\n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
    printf("\nAverage TurnAroundTime = %.2f",avgTAT/n);
    printf("\nAverage WaitingTime = %.2f",avgWT/n);
    return 0;
}
```

Result:

```
Name:Prerith M Shetty
USN:1WA24CS219
PRG:FCFS
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 2
Enter Arrival Time and Burst Time for P2: 1 2
Enter Arrival Time and Burst Time for P3: 5 3
Enter Arrival Time and Burst Time for P4: 6 4

P     ArrivalTime  BurstTime  CompletionTime  TurnAroundTime  WaitingTime  ResponseTime
1     0            2          2               2               0            0
2     1            2          4               3               1            1
3     5            3          8               3               0            0
4     6            4          12              6               2            2

Average TurnAroundTime = 3.50
Average WaitingTime = 0.75
Process returned 0 (0x0)   execution time : 32.358 s
Press any key to continue.
```

Code:

=>SJF(Non-preemptive):

```
#include <stdio.h>
int main() {
   printf("Name:Prerith M Shetty\n" "USN:1WA24CS219\n"
   "PRG:SJF\n"); int n, i, pos;
   printf("Enter number of processes: ");
   scanf("%d",&n);

   int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n], done[n];
   int current_time = 0, completed = 0;

   float avgTAT=0, avgWT=0;

   for(i=0;i<n;i++){
      printf("Enter AT and BT for P%d: ",i+1);
      scanf("%d %d",&AT[i],&BT[i]);
      done[i]=0;
   }

   while(completed < n){
      int minBT = 999;

      for(i=0;i<n;i++){
         if(AT[i] <= current_time && done[i]==0 && BT[i] < minBT){
            minBT = BT[i];
            pos = i;
         }
      }
```

```
        if(minBT == 999){
            current_time++;
        }
        else{
            RT[pos] = current_time - AT[pos];
            CT[pos] = current_time + BT[pos];
            TAT[pos] = CT[pos] - AT[pos];
            WT[pos] = TAT[pos] - BT[pos];

            current_time = CT[pos];
            done[pos] = 1;
            completed++;
            avgTAT += TAT[pos];
            avgWT += WT[pos];
        }
    }

    printf("\n%-5s %-12s %-10s %-15s %-15s %-12s %-12s\n",

"P","ArrivalTime","BurstTime","CompletionTime","TurnAroundTime","WaitingTime","Response
Time");

    for(i=0;i<n;i++)
        printf("%-5d %-12d %-10d %-15d %-15d %-12d %-12d\n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);

    printf("\nAverage TurnAroundTime = %.2f",avgTAT/n);
    printf("\nAverage WaitingTime = %.2f",avgWT/n);

    return 0;
}
```

Result:

```
Name:Prerith M Shetty
USN:1WA24CS219
PRG:SRTF
Enter number of processes: 5
Enter AT and BT for P1: 2 1
Enter AT and BT for P2: 1 5
Enter AT and BT for P3: 4 1
Enter AT and BT for P4: 0 6
Enter AT and BT for P5: 2 3

P     ArrivalTime  BurstTime  CompletionTime  TurnAroundTime  WaitingTime  ResponseTime
1     2           1          3               1               0            0
2     1           5          11              10              5            0
3     4           1          5               1               0            0
4     0           6          16              16              10           0
5     2           3          7               5               2            1

Average TurnAroundTime = 6.60
Average WaitingTime = 3.40
Process returned 0 (0x0)   execution time : 27.773 s
Press any key to continue.
```

Code:

=>SJF (preemptive) / SRTF

```
#include <stdio.h>
int main() {
    printf("Name:Prerith M Shetty\n"
    "USN:1WA24CS219\n""PRG:SRTF\n"); int n;
    printf("Enter number of processes: ");
    scanf("%d",&n);

    int AT[n], BT[n], RT[n], CT[n], TAT[n], WT[n], start[n];
    int i, time = 0, completed = 0, min, pos;
    float avgTAT=0, avgWT=0;
```

```
    for(i=0;i<n;i++){
        printf("Enter AT and BT for P%d: ",i+1)

scanf("%d %d",&AT[i],&BT[i]);

        RT[i] = BT[i];
        start[i] = -1;
    }
    while(completed < n){
        min = 999;

        for(i=0;i<n;i++){
            if(AT[i] <= time && RT[i] > 0 && RT[i] < min){
                min = RT[i];
                pos = i;
            }
        }

        if(min == 999){
            time++;
        }
        else{
            if(start[pos] == -1)
                start[pos] = time;

            RT[pos]--;
            time++;

            if(RT[pos] == 0){
                completed++;
                CT[pos] = time;
                TAT[pos] = CT[pos] - AT[pos];
                WT[pos] = TAT[pos] - BT[pos];
```

```
            avgTAT += TAT[pos];
            avgWT += WT[pos];

        }
    }
}

    printf("\n%-5s %-12s %-10s %-15s %-15s %-12s %-12s\n",
"P","ArrivalTime","BurstTime","CompletionTime","TurnAroundTime","WaitingTime","Response
Time");

    for(i=0;i<n;i++)
        printf("%-5d %-12d %-10d %-15d %-15d %-12d %-12d\n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], start[i]-AT[i]);

    printf("\nAverage TurnAroundTime = %.2f",avgTAT/n);
    printf("\nAverage WaitingTime = %.2f",avgWT/n);
    return 0;
}
```

Result:

```
Name:Prerith M Shetty
USN:1WA24CS219
PRG:SRTF
Enter number of processes: 5
Enter AT and BT for P1: 2 1
Enter AT and BT for P2: 1 5
Enter AT and BT for P3: 4 1
Enter AT and BT for P4: 0 6
Enter AT and BT for P5: 2 3

P     ArrivalTime   BurstTime   CompletionTime   TurnAroundTime   WaitingTime   ResponseTime
1     2             1           3                1                0             0
2     1             5           11               10               5             0
3     4             1           5                1                0             0
4     0             6           16               16               10            0
5     2             3           7                5                2             1

Average TurnAroundTime = 6.60
Average WaitingTime = 3.40
Process returned 0 (0x0)    execution time : 27.773 s
Press any key to continue.
```

Question:

2. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→Priority (pre-emptive & Non-pre-emptive)

→Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

=> Priority (pre-emptive)

```
#include <stdio.h>
int main() {
   printf("Name: Prerith M Shetty \nUSN:1WA24CS219\n\n");

 int n, curr_time = 0, pid[10], at[10], bt[10], remt[10], p[10], ct[10],
       tat[10], wt[10], st[10], rt[10], i, j, completed = 0, choice;
 double tat_sum = 0, wt_sum = 0;

 printf("Enter no. of processes: ");
 scanf("%d", &n);

 printf("Enter arrival times: ");
 for (i = 0; i < n; i++) {
 scanf("%d", &at[i]);
 }

 printf("Enter burst times: ");
 for (i = 0; i < n; i++) {
 scanf("%d", &bt[i]);
  remt[i] = bt[i];
 }

 printf("Enter priorities: ");
```

```
 for (i = 0; i < n; i++) {
  scanf("%d", &p[i]);
  pid[i] = i + 1;
 }

 printf("\nType:\n");
 printf("1. Lower the number higher the priority\n");
 printf("2. Higher the number higher the priority\n");
 printf("Choice: ");
 scanf("%d", &choice);

 while (completed != n)
{
   int min = 99999999,
       max = -99999999,
       s = -1;

   for (i = 0; i < n; i++) {
     if (choice == 1) {
       if (at[i] <= curr_time && p[i] < min && remt[i] > 0) {
         min = p[i];
         s = i;
       }
     }
     if (choice == 2) {
       if (at[i] <= curr_time && p[i] > max && remt[i] > 0) {
         max = p[i];
         s = i;
       }
     }
   }
```

```
  // CPU idle time
  if (s == -1) {
  curr_time++;
  continue;
  }

  if (remt[s] == bt[s])
    st[s] = curr_time;

  remt[s]--;

  if (remt[s] == 0) {
    ct[s] = curr_time + 1;
    tat[s] = ct[s] - at[s];
    tat_sum += tat[s];
    wt[s] = tat[s] - bt[s];
    wt_sum += wt[s];
    rt[s] = st[s] - at[s];
    completed++;
  }
  curr_time++;
}
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s %-6s\n",
       "P", "AT", "BT", "PR", "CT", "TAT", "WT", "RT");

for (j = 1; j < n + 1; j++) {
  for (i = 0; i < n; i++) {
    if (pid[i] == j)
      printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d %-6d\n",
             pid[i], at[i], bt[i], p[i],
```

```
            ct[i], tat[i], wt[i], rt[i]);
    }
  }

  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));
  return 0;
}
```

Result:

```
Name: Prerith M Shetty
USN:1WA24CS219

Enter no. of processes: 7
Enter arrival times: 0 1 3 4 5 6 10
Enter burst times: 8 2 4 1 6 5 1
Enter priorities: 3 4 4 5 2 6 1

Type:
1. Lower the number higher the priority
2. Higher the number higher the priority
Choice: 1

P       AT      BT      PR      CT      TAT     WT      RT
P1      0       8       3       15      15      7       0
P2      1       2       4       17      16      14      14
P3      3       4       4       21      18      14      14
P4      4       1       5       22      18      17      17
P5      5       6       2       12      7       1       0
P6      6       5       6       27      21      16      16
P7      10      1       1       11      1       0       0

Average TAT: 13.71
Average WT: 9.86
```

Code:

=>priority (Non-preemptive)

```
#include <stdio.h>

int main() {
  printf("Name: Prerith M Shetty \nUSN:1WA24CS219\n\n");

  int n, curr_time = 0, pid[10], at[10], bt[10], p[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, completed = 0, done[10] = {0}, choice;
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter AT BT Priority for process[%d]: ", i + 1);
    scanf("%d %d %d", &at[i], &bt[i], &p[i]);
  }

  printf("\nType:\n");
  printf("1. Lower the number higher the priority\n");
  printf("2. Higher the number higher the priority\n");
  printf("Choice: ");
  scanf("%d", &choice);

  while (completed != n) {

  int min = 99999999,
```

```
    max = -99999999,
    s = -1;

for (i = 0; i < n; i++) {
  // 1. AT is now or before current time
  // 2. priority is lesser than min
  // 3. process is not completed
  if (choice == 1) {
    if (at[i] <= curr_time && p[i] < min && !done[i]) {
      min = p[i];
      s = i;
    }
  }
  if (choice == 2) {
    if (at[i] <= curr_time && p[i] > max && !done[i]) {
      max = p[i];
      s = i;
    }
  }
}

// CPU idle time
if (s == -1) {
curr_time++;
continue;
}

st[s] = curr_time;
curr_time += bt[s];

// CT
ct[s] = curr_time;
```

```
  // TAT
  tat[s] = ct[s] - at[s];
  tat_sum += tat[s];

  // WT
  wt[s] = tat[s] - bt[s];
  wt_sum += wt[s];

  // RT
  rt[s] = st[s] - at[s];

  // increment the completed processes count
  completed++;
  done[s] = 1;
}

// header row
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s %-6s\n",
       "P", "AT", "BT", "PR", "CT", "TAT", "WT", "RT");

// sorted by PID
for (j = 1; j < n + 1; j++) {
  for (i = 0; i < n; i++) {
    if (pid[i] == j)
      printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d %-6d\n",
             pid[i], at[i], bt[i], p[i],
             ct[i], tat[i], wt[i], rt[i]);
  }
}

printf("\nAverage TAT: %.2f\n", (tat_sum / n));
```

```
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Result:

```
Name: Prerith M Shetty
USN:1WA24CS219

Enter no. of processes: 5
Enter AT BT Priority for process[1]: 0 3 5
Enter AT BT Priority for process[2]: 2 2 3
Enter AT BT Priority for process[3]: 3 5 2
Enter AT BT Priority for process[4]: 4 4 4
Enter AT BT Priority for process[5]: 6 1 1

Type:
1. Lower the number higher the priority
2. Higher the number higher the priority
Choice: 1

P       AT      BT      PR      CT      TAT     WT      RT
P1      0       3       5       3       3       0       0
P2      2       2       3       11      9       7       7
P3      3       5       2       8       5       0       0
P4      4       4       4       15      11      7       7
P5      6       1       1       9       3       2       2

Average TAT: 6.20
Average WT: 3.20
```

Code:

=>Round Robin

```
#include <stdio.h>

int main() {
  printf("Name: Prerith M Shetty \nUSN:1WA24CS219\n\n");

  int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10], remt[10],
        ct[10], tat[10], wt[10], st[10], rt[10], i, j, tq, q[10], f = 0,
        r = -1, inq[10] = {0};
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  printf("Enter arrival times: ");
  for (i = 0; i < n; i++) {
  scanf("%d", &at[i]);
  }

  printf("Enter burst times: ");
  for (i = 0; i < n; i++) {
  scanf("%d", &bt[i]);
    remt[i] = bt[i]; // initialising remaining time
    pid[i] = i + 1;
  }
  printf("Enter time quantum: ");
  scanf("%d", &tq);

  while (completed != n) {
```

```
for (i = 0; i < n; i++) {
  if (at[i] <= curr_time && remt[i] > 0 && !inq[i]) {
    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
  }
}

// CPU idle time
if (f > r) {
curr_time++;
continue;
}

// dequeue
i = q[f];
f = (f + 1) % 10;
inq[i] = 0;

// if the process is in the CPU for the first time
if (remt[i] == bt[i])
  st[i] = curr_time;

// decrement the remaining time of the process
if (remt[i] < tq) {
  curr_time += remt[i];
  remt[i] = 0;
} else {
  curr_time += tq;
  remt[i] -= tq;
}
```

```
  for (j = 0; j < n; j++) {
    if (at[j] <= curr_time && remt[j] > 0 && !inq[j] && j != i) {
      r = (r + 1) % 10;
      q[r] = j;
      inq[j] = 1;
    }
  }
  if (remt[i] == 0) {

    ct[i] = curr_time;

    tat[i] = ct[i] - at[i];
    tat_sum += tat[i];

    wt[i] = tat[i] - bt[i];
    wt_sum += wt[i];

    rt[i] = st[i] - at[i];

    completed++;
  } else {
    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
  }
}
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s\n",
       "P", "AT", "BT", "CT", "TAT", "WT", "RT");

// sorted by PID
for (j = 1; j < n + 1; j++) {
```

```
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d\n",
               pid[i], at[i], bt[i], ct[i],
               tat[i], wt[i], rt[i]);
    }
  }
  printf("\nAverage TAT : %.2f\n", (tat_sum / n));
  printf("Average WT  : %.2f\n", (wt_sum / n));

  return 0;
}
```

Result:

```
Name: Prerith M Shetty
USN:1WA24CS219

Enter no. of processes: 5
Enter arrival times: 0 1 2 3 4
Enter burst times: 5 3 1 2 3
Enter time quantum: 2

P       AT      BT      CT      TAT     WT      RT
P1      0       5       13      13      8       0
P2      1       3       12      11      8       1
P3      2       1       5       3       2       2
P4      3       2       9       6       4       4
P5      4       3       14      10      7       5

Average TAT : 8.60
Average WT  : 5.80
```

3. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>
int main() {
    printf("Name:Prerith M Shetty\nUSN:1WA24CS219\nPRG:MLQ (User RR + System FCFS)\n");

    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], AT[n], BT[n], type[n];
    int CT[n], TAT[n], WT[n], RT[n], remBT[n];
    int completed[n];

    for(int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT, BT and Queue No (1-User, 2-System) for P%d: ", i+1);
        scanf("%d %d %d", &AT[i], &BT[i], &type[i]);

        remBT[i] = BT[i];
        completed[i] = 0;
        RT[i] = -1;
    }

    printf("Enter Time Quantum (for User Queue): ");
    scanf("%d", &tq);
```

```
int current_time = 0, done = 0;

while(done < n) {

    int executed = 0;

    // USER QUEUE (RR - Highest Priority)
    for(int i = 0; i < n; i++) {
        if(type[i] == 1 && AT[i] <= current_time && remBT[i] > 0) {

            if(RT[i] == -1)
                RT[i] = current_time - AT[i];

            if(remBT[i] > tq) {
                current_time += tq;
                remBT[i] -= tq;
            } else {
                current_time += remBT[i];
                remBT[i] = 0;

                CT[i] = current_time;
                TAT[i] = CT[i] - AT[i];
                WT[i] = TAT[i] - BT[i];

                completed[i] = 1;
                done++;
            }

            executed = 1;
        }
    }
```

```
    //  SYSTEM QUEUE (FCFS - Lower Priority)
    if(executed == 0) {
        for(int i = 0; i < n; i++) {
            if(type[i] == 2 && AT[i] <= current_time && remBT[i] > 0) {

                if(RT[i] == -1)
                    RT[i] = current_time - AT[i];

                current_time += remBT[i];
                remBT[i] = 0;

                CT[i] = current_time;
                TAT[i] = CT[i] - AT[i];
                WT[i] = TAT[i] - BT[i];

                completed[i] = 1;
                done++;

                executed = 1;
                break; // FCFS → one at a time
            }
        }
    }

    // If CPU idle
    if(executed == 0)
        current_time++;
}

float avgTAT = 0, avgWT = 0;
printf("\n%-5s %-10s %-10s %-10s %-15s %-15s %-12s %-12s\n",
       "P","AT","BT","Queue","CT","TAT","WT","RT");
```

```
    for(int i = 0; i < n; i++) {
        printf("P%-4d %-10d %-10d %-10s %-15d %-15d %-12d %-12d\n",
               pid[i], AT[i], BT[i],
               (type[i]==1?"User":"System"),
               CT[i], TAT[i], WT[i], RT[i]);

        avgTAT += TAT[i];
        avgWT += WT[i];
    }

    printf("\nAverage Turnaround Time = %.2f", avgTAT/n);
    printf("\nAverage Waiting Time = %.2f\n", avgWT/n);

    return 0;
}
```

Result:

```
Name:Prerith M Shetty
USN:1WA24CS219
PRG:MLQ (User RR + System FCFS)
Enter number of processes: 4
Enter AT, BT and Queue No (1-User, 2-System) for P1: 0 4 1
Enter AT, BT and Queue No (1-User, 2-System) for P2: 0 3 1
Enter AT, BT and Queue No (1-User, 2-System) for P3: 0 8 2
Enter AT, BT and Queue No (1-User, 2-System) for P4: 10 5 1
Enter Time Quantum (for User Queue): 2

P       AT        BT        Queue       CT        TAT       WT        RT
P1      0         4         User        6         6         2         0
P2      0         3         User        7         7         4         2
P3      0         8         System      15        15        7         7
P4      10        5         User        20        10        5         5

Average Turnaround time = 9.50
Average Waiting Time = 4.50
```

4. Write a C program to simulate Real-Time CPU Scheduling algorithms:
→ Rate- Monotonic
→ Earliest-deadline First
→ Proportional scheduling

Code:

=> Rate- Monotonic Scheduling (RMS)

```
#include <stdio.h>
#include <math.h>

// GCD
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

// LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    printf("Name:Prerith M Shetty\n" "USN:1WA24CS219\n" "PRG:Rate monotonic scheduling (RMS) Algorithm\n");
    int n, i;

    printf("Enter the number of processes:");
```

```
scanf("%d", &n);

int bt[n], p[n];

printf("Enter the CPU burst times:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &bt[i]);
}

printf("Enter the time periods:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &p[i]);
}

// LCM
int hyper = p[0];
for (i = 1; i < n; i++) {
    hyper = lcm(hyper, p[i]);
}

printf("LCM=%d\n\n", hyper);

printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, bt[i], p[i]);
}

// Utilization
float U = 0;
for (i = 0; i < n; i++) {
    U += (float)bt[i] / p[i];
```

```
}

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n",
      U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n", hyper);

// Remaining time
int rem[n];
for (i = 0; i < n; i++)
   rem[i] = 0;

printf("\n");

// Scheduling (PRINT ONLY WHEN CHANGE HAPPENS)
int prev = -2;

for (int time = 0; time < hyper; time++) {

   // Release
   for (i = 0; i < n; i++) {
      if (time % p[i] == 0) {
         rem[i] = bt[i];
      }
   }

   int selected = -1;
   for (i = 0; i < n; i++) {
      if (rem[i] > 0) {
         if (selected == -1 || p[i] < p[selected]) {
```

```
                selected = i;
            }
        }
    }

    if (selected != prev) {
        if (selected == -1)
            printf("%dms onwards: CPU is idle\n", time);
        else
            printf("%dms onwards: Process %d running\n", time, selected + 1);
    }

    if (selected != -1) {
        rem[selected]--;
    }
    prev = selected;
  }

  return 0;
}
```

Result:

```
Name:Prerith M Shetty
USN:1WA24CS219
PRG:Rate monotonic scheduling (RMS) Algorithm
Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 32.064 s
```

Code:

=>Earliest Deadline First (EDF)

```
#include <stdio.h>

int main() {
   printf("Name:Prerith M Shetty\n" "USN:1WA24CS219\n" "PRG:Earliest Deadline first (EDF) Algorithm\n");
   int n, i;

   printf("Enter number of tasks: ");
   scanf("%d", &n);

   int bt[n], dl[n], p[n];
   int rem[n], abs_dl[n];

   printf("Enter Burst times:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
      rem[i] = 0;
   }

   printf("Enter Deadlines:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &dl[i]);
   }

   printf("Enter Periods:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &p[i]);
   }
```

```
// Initialize absolute deadlines
for (i = 0; i < n; i++) {
    abs_dl[i] = dl[i];
}

// Total time
int total_time = p[0];
for (i = 1; i < n; i++) {
    if (p[i] > total_time)
        total_time = p[i];
}

printf("\nScheduling occurs for %d ms\n\n", total_time);

int prev_idle = 0;

for (int time = 0; time < total_time; time++) {

    // Periodic release + update absolute deadline
    for (i = 0; i < n; i++) {
        if (time % p[i] == 0) {
            rem[i] = bt[i];
            abs_dl[i] = time + dl[i];
        }
    }

    int selected = -1;
    int min_deadline = 9999;

    // Select earliest absolute deadline
    for (i = 0; i < n; i++) {
        if (rem[i] > 0) {
```

```
            if (abs_dl[i] < min_deadline) {
                min_deadline = abs_dl[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        if (!prev_idle) {
            printf("%dms onwards : CPU is idle.\n", time);
            prev_idle = 1;
        }
    } else {
        printf("%dms : Task %d is running.\n", time, selected + 1);
        rem[selected]--;
        prev_idle = 0;
    }
  }

  return 0;
}
```

Result:

```
Name: Prerith M Shetty
USN:1WA24CS219
PRG:Earliest Deadline first (EDF) Algorithm
Enter number of tasks: 3
Enter Burst times:
3 2 2
Enter Deadlines:
7 4 8
Enter Periods:
20 5 10

Scheduling occurs for 20 ms

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms onwards : CPU is idle.
10ms : Task 2 is running.
11ms : Task 2 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms onwards : CPU is idle.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms onwards : CPU is idle.

Process returned 0 (0x0)   execution time : 33.078 s
```

Code:

=> Proportional Share Scheduling

```
#include  <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    printf("Name:Prerith M Shetty\n" "USN:1WA24CS219\n" "PRG:Proportional share
    scheduling\n"); int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");
    scanf("%d", &n);

    Process p[n];
    srand(time(NULL));

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);

        printf("Tickets: ");
        scanf("%d", &p[i].tickets);

        total_tickets += p[i].tickets;
```

```
    total_T += p[i].tickets;
}

printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;

        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n",
           winning_ticket, p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
           p[i].name,
           ((p[i].tickets / total_T) * 100));
}
return 0;
```

}

Result:

```
Name: Prerith M Shetty
USN:1WA24CS219
PRG:Proportional share scheduling
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 8, Winner: P1
Tickets picked: 56, Winner: P3
Tickets picked: 33, Winner: P3
Tickets picked: 31, Winner: P3
Tickets picked: 59, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 24.615 s
```

5. Write a C program to simulate

→ Producer-Consumer problem using semaphores.

→ Dining-Philosopher's problem

Code:

=>Producer-Consumer problem using semaphores.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int id = *(int*)arg;
    int item = 0;

    while (1) {
        item++;

        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer %d produced %d at buffer[%d]\n", id, item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    int item;

    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer %d consumed %d from buffer[%d]\n", id, item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }
}
```

```
int main() {
    pthread_t prod[NUM_PRODUCERS], cons[NUM_CONSUMERS];
    int p_id[NUM_PRODUCERS], c_id[NUM_CONSUMERS];

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    // Create producer threads
    for (int i = 0; i < NUM_PRODUCERS; i++) {
        p_id[i] = i + 1;
        pthread_create(&prod[i], NULL, producer, &p_id[i]);
    }

    // Create consumer threads
    for (int i = 0; i < NUM_CONSUMERS; i++) {
        c_id[i] = i + 1;
        pthread_create(&cons[i], NULL, consumer, &c_id[i]);
    }

    for (int i = 0; i < NUM_PRODUCERS; i++)
        pthread_join(prod[i], NULL);

    for (int i = 0; i < NUM_CONSUMERS; i++)
        pthread_join(cons[i], NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Result:

```
Producer 1 produced 1 at buffer[0]
Consumer 2 consumed 1 from buffer[0]
Producer 2 produced 1 at buffer[1]
Consumer 1 consumed 1 from buffer[1]
Producer 2 produced 2 at buffer[2]
Producer 1 produced 2 at buffer[3]
Consumer 1 consumed 2 from buffer[2]
Consumer 2 consumed 2 from buffer[3]
Producer 1 produced 3 at buffer[4]
Producer 2 produced 3 at buffer[0]
Producer 2 produced 4 at buffer[1]
Producer 1 produced 4 at buffer[2]
Consumer 1 consumed 3 from buffer[4]
Consumer 2 consumed 3 from buffer[0]
Producer 1 produced 5 at buffer[3]
Producer 2 produced 5 at buffer[4]
Producer 2 produced 6 at buffer[0]
Consumer 2 consumed 4 from buffer[1]
Consumer 1 consumed 4 from buffer[2]
Producer 1 produced 6 at buffer[1]
Producer 2 produced 7 at buffer[2]
Consumer 1 consumed 5 from buffer[3]
Consumer 2 consumed 5 from buffer[4]
Producer 1 produced 7 at buffer[3]
Producer 2 produced 8 at buffer[4]
Consumer 2 consumed 6 from buffer[0]
Consumer 1 consumed 6 from buffer[1]
Producer 2 produced 9 at buffer[0]
Producer 1 produced 8 at buffer[1]
Consumer 2 consumed 7 from buffer[2]
Consumer 1 consumed 7 from buffer[3]
Producer 1 produced 9 at buffer[2]
Producer 2 produced 10 at buffer[3]
Consumer 1 consumed 8 from buffer[4]
Consumer 2 consumed 9 from buffer[0]
Producer 1 produced 10 at buffer[4]
Producer 2 produced 11 at buffer[0]
Consumer 1 consumed 8 from buffer[1]
Producer 2 produced 12 at buffer[1]
Consumer 2 consumed 9 from buffer[2]
Producer 1 produced 11 at buffer[2]
```

Code:

=> Dining-Philosopher’s problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;                  // left fork index
    int right = (id + 1) % N;       // right fork index

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking

        // Deadlock avoidance
        if (id % 2 == 0)
        {
            // Pick left then right
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
```

```
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
            // Pick right then left
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]);  // FIXED LINE
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main()
{
    int i;
    int ids[N];

    // Initialize forks
```

```
    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i],  NULL);
    }

    // Destroy mutexes
    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Result:

```
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 4 picked up left fork 4.
Philosopher 0 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 4 put down forks 4 and 0.
Philosopher 0 is eating.
Philosopher 4 is thinking.
Philosopher 0 put down forks 0 and 1.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
```

6. Write a C program to simulate:

→ Bankers' algorithm for the purpose of deadlock avoidance.

→ Deadlock Detection

Code:

=> Bankers' algorithm for the purpose of deadlock avoidance.

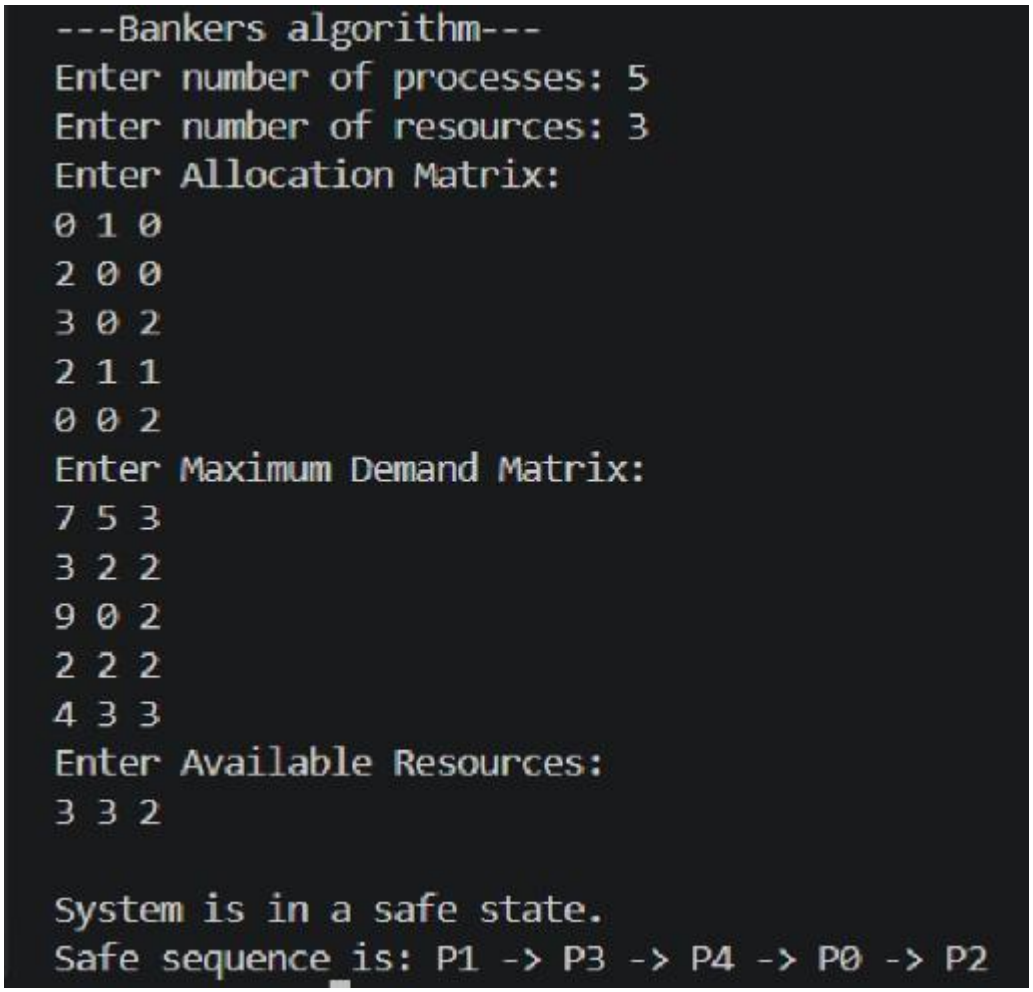
```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("---Bankers algorithm--- \n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    // Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Maximum Matrix
    printf("Enter Maximum Demand Matrix:\n");
```

```
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}

// Available Resources
printf("Enter Available Resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

// Need Matrix
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n], work[m];

for(i = 0; i < n; i++)
    finish[i] = 0;

for(i = 0; i < m; i++)
    work[i] = avail[i];

int count = 0;

while(count < n) {

    int found = 0;
```

```
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            int possible = 1;
            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }

            if(possible) {
                for(k = 0; k < m; k++)
                    work[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}
if(count == n) {

    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
```

```
            if(i != n - 1)
                printf(" -> ");
        }
        printf("\n");
    }
    else {
        printf("\nSystem is NOT in a safe state.\n");
    }
    return 0;
}
```

Result:



Code:

=> Deadlock Detection

```
#include <stdio.h>

int main() {

   int n, m, i, j, k;
   printf("\n---Deadlock Detection algorithm--- \n");
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   printf("Enter the number of resources: ");
   scanf("%d", &m);

   int allocation[n][m];
   int request[n][m];
   int available[m];

   // Allocation Matrix
   printf("\nEnter the allocation matrix:\n");

   for(i = 0; i < n; i++) {

      printf("Process %d: ", i);

      for(j = 0; j < m; j++) {
         scanf("%d", &allocation[i][j]);
      }
   }
```

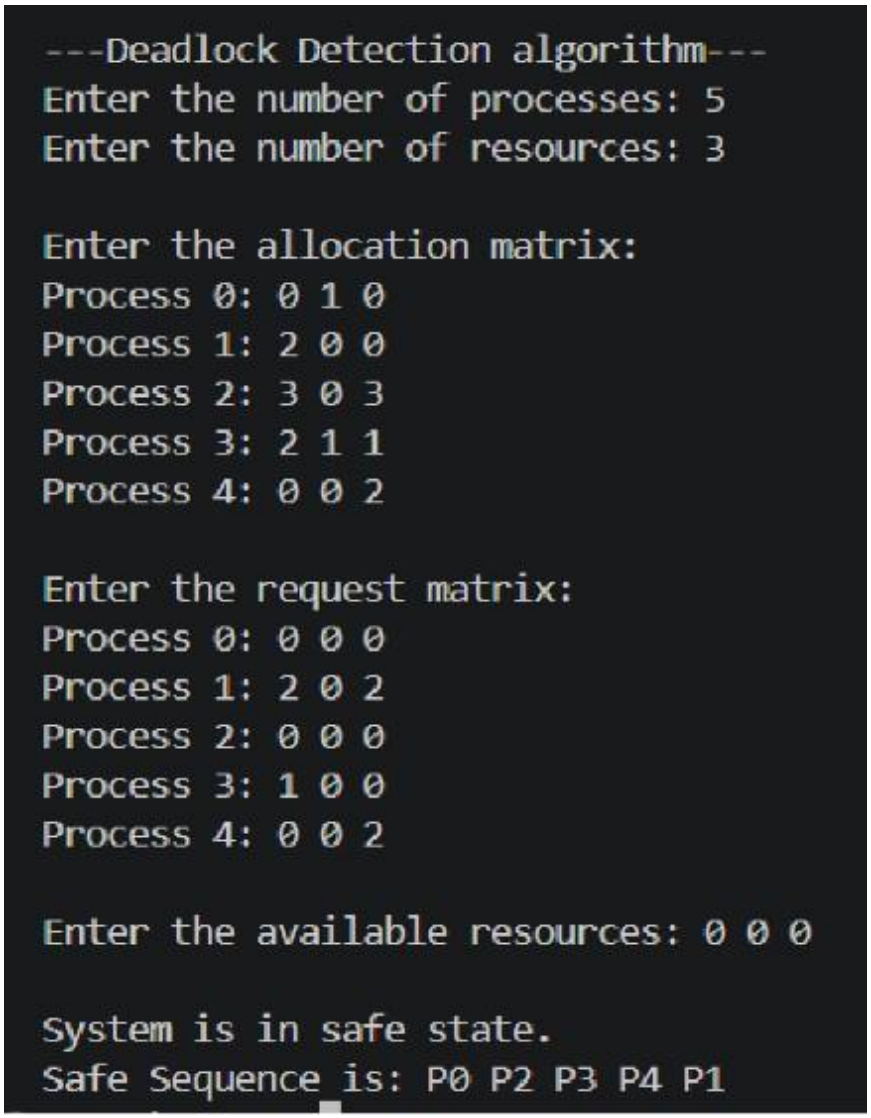
```
// Request Matrix
printf("\nEnter the request matrix:\n");

for(i = 0; i < n; i++) {

   printf("Process %d: ", i);

   for(j = 0; j < m; j++) {
      scanf("%d", &request[i][j]);
   }
}

// Available Resources
printf("\nEnter the available resources: ");

for(i = 0; i < m; i++) {
   scanf("%d", &available[i]);
}

int work[m];
int finish[n];
int safeSeq[n];

// Initialize
for(i = 0; i < m; i++)
   work[i] = available[i];

for(i = 0; i < n; i++) {
   int zero = 1;

   for(j = 0; j < m; j++) {
      if(allocation[i][j] != 0) {
```

```
            zero = 0;
            break;
        }
    }

    if(zero)
        finish[i] = 1;
    else
        finish[i] = 0;
}
int count = 0;

while(count < n) {
    int found = 0;

    for(i = 0; i < n; i++) {

        if(finish[i] == 0) {

            int possible = 1;

            for(j = 0; j < m; j++) {
                if(request[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }

            if(possible) {
                for(k = 0; k < m; k++)
                    work[k] += allocation[i][k];
                safeSeq[count++] = i;
```

```
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0)
        break;
}
int deadlock = 0;
for(i = 0; i < n; i++) {
    if(finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

if(deadlock == 0) {
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < count; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");
}
else {
    printf("\nSystem is in deadlock state.\n");
    printf("Deadlocked processes are: ");

    for(i = 0; i < n; i++) {
        if(finish[i] == 0)
            printf("P%d ", i);
```

```
        }
        printf("\n");
    }

    return 0;
}
```

Result:



7. Write a C program to simulate the following contiguous memory allocation techniques.

→ Worst-fit

→ Best-fit

→ First-fit

Code:

```
#include <stdio.h>

#define MAX 100

void firstFit(int blockSize[], int originalBlock[], int blocks,
              int processSize[], int processes) {

   int allocation[MAX];

   for(int i = 0; i < processes; i++) {
      allocation[i] = -1;
   }

   for(int i = 0; i < processes; i++) {

      for(int j = 0; j < blocks; j++) {

         if(blockSize[j] >= processSize[i]) {

            allocation[i] = j;

            // Remaining memory reused
            blockSize[j] -= processSize[i];

            break;
         }
```

```
        }
    }

    printf("\n      ---------- FIRST FIT---------- \n");

    printf("Process No\tProcess Size\tBlock No\tBlock Size\n");

    for(int i = 0; i < processes; i++) {

        printf("%d\t\t%d\t\t",
              i + 1, processSize[i]);

        if(allocation[i] != -1) {

            printf("%d\t\t%d\n",
                  allocation[i] + 1,
                  originalBlock[allocation[i]]);
        }
        else {

            printf("Not Allocated\n");
        }
    }
}

void bestFit(int blockSize[], int originalBlock[], int blocks,
             int processSize[], int processes) {

    int allocation[MAX];

    for(int i = 0; i < processes; i++) {
        allocation[i] = -1;
```

```
}

for(int i = 0; i < processes; i++) {

   int bestIdx = -1;

   for(int j = 0; j < blocks; j++) {

      if(blockSize[j] >= processSize[i]) {

         if(bestIdx == -1 ||
           blockSize[j] < blockSize[bestIdx]) {

            bestIdx = j;
         }
      }
   }

   if(bestIdx != -1) {

      allocation[i] = bestIdx;

      // Remaining memory reused
      blockSize[bestIdx] -= processSize[i];
   }
}

printf("\n      ---------- BEST FIT ----------\n");

printf("Process No\tProcess Size\tBlock No\tBlock Size\n");

for(int i = 0; i < processes; i++) {
```

```
        printf("%d\t\t%d\t\t",
              i + 1, processSize[i]);

        if(allocation[i] != -1) {

           printf("%d\t\t%d\n",
                 allocation[i] + 1,
                 originalBlock[allocation[i]]);
        }
        else {

           printf("Not Allocated\n");
        }
    }
}

void worstFit(int blockSize[], int originalBlock[], int blocks,
            int processSize[], int processes) {

    int allocation[MAX];

    for(int i = 0; i < processes; i++) {
        allocation[i] = -1;
    }

    for(int i = 0; i < processes; i++) {

        int worstIdx = -1;

        for(int j = 0; j < blocks; j++) {
```

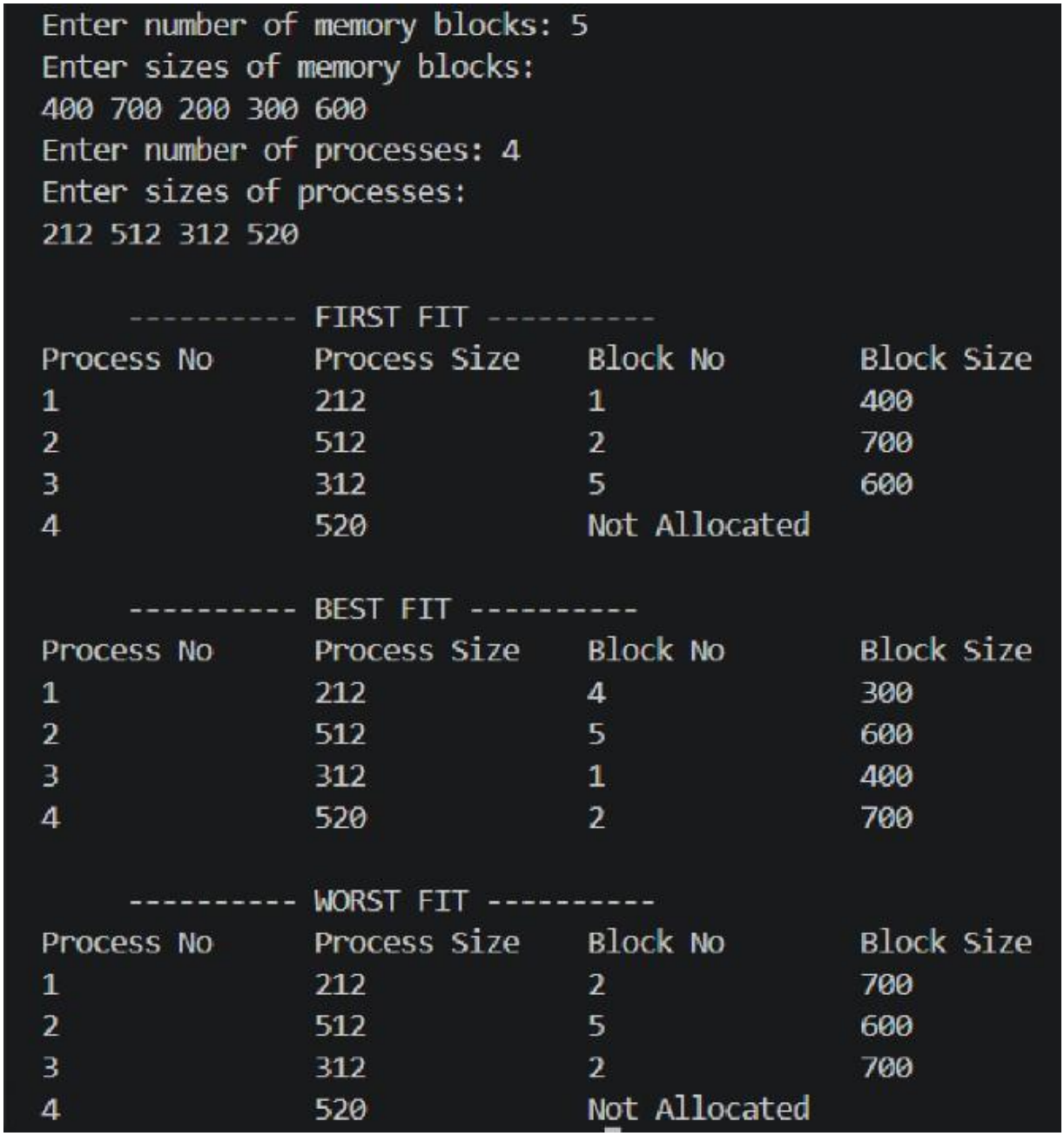
```
        if(blockSize[j] >= processSize[i]) {

            if(worstIdx == -1 ||
              blockSize[j] > blockSize[worstIdx]) {

                worstIdx = j;
            }
        }
    }

    if(worstIdx != -1) {

        allocation[i] = worstIdx;

        // Remaining memory reused
        blockSize[worstIdx] -= processSize[i];
    }
}

printf("\n      ---------- WORST FIT ----------- \n");

printf("Process No\tProcess Size\tBlock No\tBlock Size\n");

for(int i = 0; i < processes; i++) {

    printf("%d\t\t%d\t\t",
          i + 1, processSize[i]);

    if(allocation[i] != -1) {

        printf("%d\t\t%d\n",
              allocation[i] + 1,
```

```
                originalBlock[allocation[i]]);
        }
        else {

            printf("Not Allocated\n");
        }
    }
}

int main() {

    int blocks, processes;

    int blockSize1[MAX], blockSize2[MAX], blockSize3[MAX];
    int originalBlock[MAX];
    int processSize[MAX];

    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);

    if(blocks <= 0 || blocks > MAX) {
        printf("Invalid number of blocks!\n");
        return 0;
    }

    printf("Enter sizes of memory blocks:\n");
    for(int i = 0; i < blocks; i++) {
        scanf("%d", &blockSize1[i]);
        if(blockSize1[i] <= 0) {
            printf("Invalid block size!\n");
            return 0;
        }
```

```
    originalBlock[i] = blockSize1[i];
    blockSize2[i] = blockSize1[i];
    blockSize3[i] = blockSize1[i];
}

printf("Enter number of processes: ");
scanf("%d", &processes);

if(processes <= 0 || processes > MAX) {
    printf("Invalid number of processes!\n");
    return 0;
}

printf("Enter sizes of processes:\n");
for(int i = 0; i < processes; i++) {

    scanf("%d", &processSize[i]);

    if(processSize[i] <= 0) {
        printf("Invalid process size!\n");
        return 0;
    }
}

firstFit(blockSize1, originalBlock, blocks,
         processSize, processes);

bestFit(blockSize2, originalBlock, blocks,
        processSize, processes);

worstFit(blockSize3, originalBlock, blocks,
         processSize, processes);
```

```
    return 0;
}
```

Result:



```
Enter number of memory blocks: 5
Enter sizes of memory blocks:
400 700 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 512 312 520

     ---------- FIRST FIT ----------
Process No      Process Size    Block No        Block Size
1               212             1               400
2               512             2               700
3               312             5               600
4               520             Not Allocated

     ---------- BEST FIT ----------
Process No      Process Size    Block No        Block Size
1               212             4               300
2               512             5               600
3               312             1               400
4               520             2               700

     ---------- WORST FIT ----------
Process No      Process Size    Block No        Block Size
1               212             2               700
2               512             5               600
3               312             2               700
4               520             Not Allocated
```

8. Write a C program to simulate page replacement algorithms.

→ FIFO

→ LRU

→ Optimal

Code:

```
#include <stdio.h>

int main()
{
   int pages[50], frames[10];
   int n, f;
   int i, j, k;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter page reference string:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &pages[i]);
   }

   printf("Enter number of frames: ");
   scanf("%d", &f);

   // - - - - - - - - - - FIFO - - - - - - - - - -

   int faults = 0;
   int next = 0;
   int found;

   for(i = 0; i < f; i++)
   {
      frames[i] = -1;
   }

   printf("\n--- FIFO Page Replacement ---\n");

   for(i = 0; i < n; i++)
```

```
{
    found = 0;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }

    if(found == 0)
    {
        frames[next] = pages[i];
        next = (next + 1) % f;
        faults++;
    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < f; j++)
    {
        if(frames[j] != -1)
        {
            printf("%d", frames[j]);

            if(j != f - 1)
            {
                printf(" ");
            }
        }
    }

    printf("]\n");
}

printf("Total Page Faults (FIFO): %d\n", faults);

// ---------- LRU ----------

int time[10];
int count = 0;
faults = 0;
```

```
for(i = 0; i < f; i++)
{
   frames[i] = -1;
   time[i] = 0;
}

printf("\n--- LRU Page Replacement ---\n");

for(i = 0; i < n; i++)
{
   found = 0;

   for(j = 0; j < f; j++)
   {
      if(frames[j] == pages[i])
      {
         found = 1;
         count++;
         time[j] = count;
         break;
      }
   }

   if(found == 0)
   {
      int pos = -1;

      for(j = 0; j < f; j++)
      {
         if(frames[j] == -1)
         {
            pos = j;
            break;
         }
      }

      if(pos == -1)
      {
         pos = 0;

         for(j = 1; j < f; j++)
         {
            if(time[j] < time[pos])
```

```
            {
                pos = j;
            }
        }
    }

    frames[pos] = pages[i];
    count++;
    time[pos] = count;
    faults++;
  }

  printf("Page %d -> [", pages[i]);

  for(j = 0; j < f; j++)
  {
    if(frames[j] != -1)
    {
      printf("%d", frames[j]);

      if(j != f - 1)
      {
        printf(" ");
      }
    }
  }

  printf("]\n");
}

printf("Total Page Faults (LRU): %d\n", faults);

// ---------- OPTIMAL ----------

faults = 0;

for(i = 0; i < f; i++)
{
  frames[i] = -1;
}

printf("\n--- Optimal Page Replacement ---\n");

for(i = 0; i < n; i++)
```

```
{
   found = 0;

   for(j = 0; j < f; j++)
   {
      if(frames[j] == pages[i])
      {
         found = 1;
         break;
      }
   }

   if(found == 0)
   {
      int pos = -1;

      for(j = 0; j < f; j++)
      {
         if(frames[j] == -1)
         {
            pos = j;
            break;
         }
      }

      if(pos == -1)
      {
         int farthest = -1;
         int index;

         for(j = 0; j < f; j++)
         {
            int found_future = 0;

            for(k = i + 1; k < n; k++)
            {
               if(frames[j] == pages[k])
               {
                  if(k > farthest)
                  {
                     farthest = k;
                     pos = j;
                  }
```

```
                found_future = 1;
                break;
            }
        }

        if(found_future == 0)
        {
            pos = j;
            break;
        }
    }
  }

  frames[pos] = pages[i];
  faults++;
 }

 printf("Page %d -> [", pages[i]);

 for(j = 0; j < f; j++)
 {
  if(frames[j] != -1)
  {
   printf("%d", frames[j]);

   if(j != f - 1)
   {
    printf(" ");
   }
  }
 }

 printf("]\n");
}

printf("Total Page Faults (Optimal): %d\n", faults);

return 0;
}
```

Result:

```
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7
```